Übungsblatt 1: Zeitvertrag, Polling und Interrupts

Musterlösung · Echtzeitsysteme (EZS)

Prof. Dr.-Ing. Daniel Klünder

Musterlösung · Rechenwerte sind Lehrannahmen, keine Hardwaremessungen.

Ziele und Zeitrahmen

Sie formulieren einen Zeitvertrag, berechnen Antwortzeit und Schwankung und beurteilen Polling beziehungsweise Interrupts unter benannten Annahmen. Die vier Papieraufgaben sind ohne Board lösbar und üben klausurtypische Rechnungen und Begründungen. Danach vergleichen Sie einfache Programmstrukturen auf dem **NUCLEO-H755ZI-Q** mit STM32H755ZIT6: **Cortex-M4 ohne Betriebssystem**, Cortex-M7 mit vorbereitetem generiertem Startcode.

Abschnitt	Minuten	Umfang
Papieraufgabe 1	8	8 Punkte
Papieraufgabe 2	12	8 Punkte
Papieraufgabe 3	10	8 Punkte
Papieraufgabe 4	10	8 Punkte
Vorbereiteter Programmvergleich	35	Beobachtung und Protokoll

Die 75 Minuten ergänzen 90 Minuten Vorlesung und 15 Minuten Quiz zum Termin mit 180 Minuten. Installation und erster Dual-Core-Start sind Vorbereitung. Alle folgenden Zahlen sind **didaktische Annahmen**, soweit keine eigenen Messwerte eingetragen werden. Fertigstellung auf der Deadline gilt als rechtzeitig.

Papieraufgabe 1: Vertrag und Echtzeitklasse

Ein Sensor meldet ein Werkstück. Der richtige Stellauftrag muss innerhalb von 10 ms am Ausgang anliegen. Die mechanische Klappenbewegung gehört **nicht** zu diesem Softwarevertrag. Für den Stellauftrag ist kein Miss zulässig. Ein Prüfbild desselben Geräts ist nach seiner Frist wertlos; ein fehlendes Bild je Block von 100 Bildern ist zulässig. Eine Statusanzeige besitzt auch bei verspäteter Aktualisierung noch Restnutzen.

1. Benennen Sie Beginn, Abschluss und relative Frist des Stellauftrags. Warum reicht „die Software soll schnell sein“ nicht? (2 P.)
2. Ordnen Sie die drei Aufgaben Hard, Firm und Soft zu. Begründen Sie jede Zuordnung aus dem Vertrag. (3 P.)
3. Das Sensorereignis gibt den Auftrag bei $r = 2\,\text{ms}$ frei. Berechnen Sie die absolute Deadline d. Beurteilen Sie einen richtigen Auftrag bei $f = 12\,\text{ms}$ und bei $f = 12{,}1\,\text{ms}$. (2 P.)

4. Ein fachlich falscher Stellauftrag liegt schon bei $f = 8\,\text{ms}$ vor. Ist der Vertrag erfüllt? Begründen Sie. (1 P.)

Ihre Lösung: Zeitpunkte, Klassen und kurze Begründungen auf einem separaten Blatt.

Musterlösung Papieraufgabe 1

1. Beginn ist die Freigabe durch das erfasste Sensorereignis, Abschluss der **richtige** Stellauftrag am Ausgang, relative Frist $D = 10\,\text{ms}$. „Schnell“ legt weder Endpunkte noch einen zulässigen Maximalwert fest. Je 1 P. für den vollständigen Vertrag und die Begründung.
2. Stellauftrag **Hard**: kein deadline miss, also keine Fristverletzung, erlaubt. Prüfbild **Firm**: verspätetes Einzelergebnis wertlos, zugleich eine benannte Auslassung zulässig. Anzeige **Soft**: verspäteter Stand hat Restnutzen. Je 1 P. mit Begründung. Gerätetyp oder Branche ersetzen die Zuordnung nicht.
3. $d = r + D = 2 + 10 = 12\,\text{ms}$. Bei $f = 12$ liegt der richtige Auftrag genau auf der Frist und ist nach der gewählten Konvention zulässig. Bei $f = 12{,}1$ ist er 0,1 ms verspätet. Je 1 P. für Rechnung und beide Bewertungen.
4. Nein: Die Zeitbedingung ist erfüllt, die funktionale Bedingung verletzt. Ein pünktlich falscher Wert erfüllt den Gesamtvertrag nicht. 1 P.

Papieraufgabe 2: Zeitachse, Jitter und Nachweis

Ein Auftrag wird bei $r = 2\,\text{ms}$ freigegeben und startet erstmals bei $s = 4\,\text{ms}$. Er rechnet von 4 bis 6, wird von 6 bis 8 unterbrochen und rechnet von 8 bis 9 weiter. Die absolute Deadline liegt bei $d = 12\,\text{ms}$. Verwaltungsaufwand ist im Modell nicht zusätzlich angesetzt.

1. Skizzieren Sie die Abschnitte. Berechnen Sie eigenen Rechenbedarf C, Startverzug, Antwortzeit R, relative Deadline D und Reserve. Prüfen Sie die Frist. (3 P.)
2. Eine Ausgabe hat bei Sollperiode $T = 10\,\text{ms}$ die Lehrzeitpunkte 0; 9,8; 20; 29,8; 40 ms. Berechnen Sie die vier Istperioden, deren Mittelwert und Peak-to-Peak-Periodenspanne. (2 P.)
3. Berechnen Sie die Abweichungen gegenüber den Sollzeitpunkten 0; 10; 20; 30; 40 ms und deren Spanne. Weshalb ist sie nicht der Periodenjitter aus Teil 2? (1 P.)
4. Eine andere Funktion zeigt in 100 untersuchten Läufen höchstens 80 µs eigene Ausführungszeit. Darf man deshalb WCET = 80 µs garantieren? Beweisen konstante Latenzen von je 5 ms eine Frist von 3 ms? Begründen Sie beides. (2 P.)

WCET steht für Worst-Case Execution Time. Peak-to-Peak bezeichnet Maximum minus Minimum.

Musterlösung Papieraufgabe 2

1. Die Skizze muss Warten 2–4, Rechnen 4–6, Unterbrechung 6–8 und Rechnen 8–9 zeigen. $C = (6 - 4) + (9 - 8) = 3\,\text{ms}$, Startverzug $s - r = 2\,\text{ms}$, $R = f - r = 9 - 2 = 7\,\text{ms}$, $D = d - r = 12 - 2 = 10\,\text{ms}$, Reserve $d - f = 3\,\text{ms}$. $R \leq D$: der Modelllauf ist rechtzeitig. Je 1 P. für Skizze mit Rechenbedarf, weitere Größen und Fristbewertung.
2. Istperioden: 9,8; 10,2; 9,8; 10,2 ms. Mittelwert $= (9{,}8 + 10{,}2 + 9{,}8 + 10{,}2)/4 = 10\,\text{ms}$. Spanne $= 10{,}2 - 9{,}8 = 0{,}4\,\text{ms}$. Je 1 P. für Perioden und Auswertung.

3. Phasenabweichungen: 0; −0,2; 0; −0,2; 0 ms. Spanne $= 0 - (-0{,}2) = 0{,}2\,\text{ms}$. Die Phase vergleicht Ist mit Soll desselben Ereignisses, die Periode dagegen benachbarte Istereignisse. 1 P. mit dieser Unterscheidung.
4. Nein: 80 µs ist das größte beobachtete Ergebnis; ungesehene Eingaben oder andere zugelassene Zustände können länger dauern. Eine sichere Schranke braucht eine begründete Abdeckung beziehungsweise Analyse. Auch 5 ms bei jedem Lauf verletzen $D = 3\,\text{ms}$, obwohl die beobachtete Latenzspanne null ist. Je 1 P. Eine Reserve auf das Messmaximum allein beweist ebenfalls keine Schranke.

Papieraufgabe 3: Polling und gespeicherte Ereignisse

Eine Super-Loop prüft pro Runde einmal einen Eingang (höchstens 0,1 ms), reagiert bei Auftrag (höchstens 0,2 ms) und bearbeitet die Anzeige (höchstens 5,0 ms). Ein Auftrag bleibt bis zur Abfrage gespeichert und wird im Modell jeweils am Ende der Eingangsprüfung erkannt. Es gibt höchstens einen relevanten Auftrag und keine Unterbrechungen. Für eine konservative Abschätzung begrenzen Sie das Warten bis zur Erkennung durch eine maximale ganze Runde und addieren anschließend die Reaktion. Die Frist ist $D = 10\,\text{ms}$.

1. Berechnen Sie maximale Rundenzeit und konservative Antwortgrenze. Ist die Modellfrist nachweisbar? (2 P.)
2. Die Anzeigenarbeit benötigt nun 50 ms. Rechnen Sie erneut und beschreiben Sie eine Ereignisphase, in der tatsächlich eine Fristverletzung möglich ist. (2 P.)
3. Statt des gespeicherten Auftrags wird nur der momentane Eingangspegel gelesen. Die Abfragen erfolgen bei 0, 5 und 10 ms; der Eingang ist nur von 1 bis 2 ms HIGH. Wird das Ereignis erkannt? Erklären Sie den Unterschied zwischen Verlust und Verspätung sowie die Funktion eines Latch. (2 P.)
4. Die Anzeige wird in Schritte von höchstens 1 ms zerlegt, mit einer Eingangsprüfung nach jedem Schritt. Berechnen Sie die neue Runden- und Antwortgrenze. Welche zusätzliche Zustandsinformation braucht die Anzeige? (2 P.)

Musterlösung Papieraufgabe 3

1. Runde $= 0{,}1 + 0{,}2 + 5{,}0 = 5{,}3\,\text{ms}$, konservativ $R \le 5{,}3 + 0{,}2 = 5{,}5\,\text{ms}$. Unter den gegebenen Annahmen gilt $R \le D = 10\,\text{ms}$. Je 1 P. für die Grenzen und die bedingte Fristbewertung. Die Abschätzung muss nicht die engstmögliche sein.
2. Runde $= 50{,}3\,\text{ms}$ und konservative Antwortgrenze $50{,}5\,\text{ms}$. Eine Grenze oberhalb der Frist allein ist noch kein Beweis eines Miss. Ein Auftrag unmittelbar nach seiner Prüfung beziehungsweise vor Beginn der 50-ms-Anzeigenarbeit wartet aber im angegebenen Ablauf schon annähernd 50 ms und verletzt damit 10 ms tatsächlich. Je 1 P. für Rechnung und konkrete Phase.
3. Alle drei Abfragen sehen LOW, daher wird der Puls nicht erkannt. Ein Verlust erzeugt keinen später bearbeiteten Auftrag; ein gespeicherter Auftrag kann dagegen erst verspätet bearbeitet werden. Ein Latch hält das Ereignismerkmal bis zur Abfrage fest, hat aber nicht automatisch Platz für beliebig viele Ereignisse. Je 1 P. für Erkennung und Begriffstrennung.
4. Runde $= 0{,}1 + 0{,}2 + 1{,}0 = 1{,}3\,\text{ms}$, konservativ $R \le 1{,}5\,\text{ms}$. Die Anzeige speichert ihren Fortschritt beziehungsweise den nächsten zu bearbeitenden Schritt zwischen Runden; die Gesamtanzeige braucht weiterhin mehrere Schritte. Je 1 P. für Rechnung und Zustand.

Papieraufgabe 4: Interrupts und Timer

1. Zwei Handler haben denselben Präemptionsrang 2, aber Subpriority 0 und 1. Kann der mit Subpriority 0 den laufenden anderen Handler allein deshalb unterbrechen? Was ändert eine zulässige Präemptionspriorität 1? Kleinere Zahlen sind dringender; die Gruppierung stellt die verwendeten Subpriority-Bits bereit. (2 P.)
2. Main liest ein gesetztes Flag, dann setzt ein zweites Interrupt-Ereignis es erneut, danach löscht Main das Flag. Was geht verloren? Warum lösen `volatile` und ein einzelnes Flag die vollständige Ereigniszählung nicht? (2 P.)
3. Nur für den gegebenen tatsächlichen Timer-Kerntakt $f_{\mathrm{TIM}} = 100\,\mathrm{MHz}$ seien $PSC = 9999$, $ARR = 4999$. Berechnen Sie Updatefrequenz, Updateabstand und vollständige LED-Blinkfrequenz bei einem Toggle pro Update. (2 P.)
4. Ein Handler erfasst früh, die Hauptbearbeitung startet aber erst 3 ms nach Freigabe und benötigt 1 ms. Prüfen Sie $D = 3\,\mathrm{ms}$. Ist Pin-Flanke bis HAL-Callback-Marker dasselbe wie reine Ausnahme-Eintrittszeit? (2 P.)

Verwenden Sie $f_{\mathrm{UPD}} = f_{\mathrm{TIM}}/((PSC + 1)(ARR + 1))$. PSC ist der Vorteilerregisterwert, ARR der Zählende-Registerwert. LED bedeutet Light-Emitting Diode, Leuchtdiode; HAL bedeutet Hardware Abstraction Layer, Hardware-Abstraktionsschicht.

Musterlösung Papieraufgabe 4

1. Nein. Subpriority ordnet gleichzeitig anstehende zulässige Anforderungen gleichen Präemptionsrangs; sie bewirkt keine Unterbrechung eines laufenden Handlers. Ein zulässiger Rang 1 kann Rang 2 unterbrechen. Je 1 P.
2. Nach dem Löschen bleibt keine Information über das zweite Ereignis. `volatile` erhält erforderliche Zugriffe, macht Lesen und Löschen aber nicht zu einem unteilbaren Austausch. Auch ein geschützter Austausch zählt mit einem einzigen Flag nicht automatisch mehrere Ereignisse. Für jede einzeln benötigte Information braucht man geeignete Zählung oder Pufferung mit Kapazitätsregeln. Je 1 P. für Verlust und Begründung.
3. $f_{\mathrm{UPD}} = 100 \cdot 10^6/(10000 \cdot 5000) = 2\,\mathrm{Hz}$. Updateabstand $1/2\,\mathrm{s} = 500\,\mathrm{ms}$. Zwei Toggles ergeben einen kompletten An-Aus-Zyklus von 1 s, also 1 Hz Blinkfrequenz. Je 1 P. für Update und Blinkzyklus. Die Frequenz gilt nur bei tatsächlich geprüftem Timer-Takt, nicht automatisch beim gleichen CPU-Takt.
4. $R = 3 + 1 = 4\,\mathrm{ms} > D$: Verletzung um 1 ms, obwohl die frühe Erfassung kurz ist. Pin bis Callback enthält zusätzlich unter anderem Erkennung, Weiterleitung, Maskierung und Softwareaufrufe; es ist nicht reine Eintrittszeit. Je 1 P.

Programmvergleich: vorbereitete 35 Minuten

Entwicklungsweg: **STM32CubeMX**, **VS Code mit STM32 VS Code Extension** und **NUCLEO-H755ZI-Q**. Benutzen Sie das von der Lehrperson vorbereitete Dual-Core-Projekt und die vollständigen Anwendungsbausteine in code/lab01. Deren README beschreibt Vorbereitung, Integration, Dateien und Prüfgrenzen. Es gibt kein Betriebssystem und keinen eigenen gemeinsamen Datenblock.

Vorbereitung prüfen · 5 Minuten

- Beide Firmwareziele CM4 und CM7 sind bereits gebaut, geladen und korrekt gestartet. Generierten Bootcode erhalten.

- Nur CM4 besitzt TIM2 und PE1 / gelbe LD2. Pinbelegung/Bestückung am Board prüfen.
- TIM2 hat einen geprüften Updateabstand von 500 ms und eine aktivierte CM4-Interruptkette; TIM2 ist nicht die HAL-Zeitbasis. Der Anwendungsbaustein startet TIM2 nur im Timer-Modus.
- CubeMX-, Firmwarepaket-, Erweiterungs- und Compilerstand sowie Takte festhalten. Bei nicht funktionierendem Aufbau die offene Stelle dokumentieren, keine Messergebnisse einsetzen.

Anwendung und vier Läufe · 20 Minuten

`lab01_demo.c` und den Include-Pfad nur in das tatsächliche CM4-CMake-Ziel aufnehmen. Im vorhandenen Anwender-Include-Bereich `#include "lab01_demo.h"` ergänzen. Nach generierter GPIO-/TIM2-Initialisierung:

```
if (!Lab01_Init(LAB01_BASELINE, &htim2)) {
    Error_Handler();
}
```

Im Rumpf der **vorhandenen** Endlosschleife nur einen Anwendungsschritt aufrufen:

```
Lab01_Step();
```

Den vorhandenen Timer-Callback einmal um `Lab01_TimerCallback(htim);` ergänzen. Vorhandene HAL-Zeitbasisbehandlung erhalten und keinen gleichnamigen Callback doppelt definieren. Die komplette Einbindung steht im README.

Wechseln Sie nacheinander den ersten Initialisierungsparameter zu:

1. `LAB01_BASELINE`: Main-Umschaltung, Delay ohne Zusatzlast.
2. `LAB01_POLL_CONSTANT`: feste Iterationszahl zusätzlich zum Delay.
3. `LAB01_POLL_VARIABLE`: wechselnde Iterationszahl zusätzlich zum Delay.
4. `LAB01_TIMER`: Main nur variable Last; Umschaltung ausschließlich im TIM2-Callback.

Nach jeder Änderung neu bauen und nach Reset korrekt starten. Entfernen Sie anderen LED-/Delay-Code, der den Vergleich überlagern würde. Eine Zähliteration hat keine hier bekannte Dauer in Millisekunden. Lesen Sie die Diagnosedaten `lab01_diag` nur bei angehaltenem CM4 als konsistenten Gesamtstand; Debughalt nicht in eine Timing-Aufzeichnung aufnehmen.

Protokoll und Interpretation · 10 Minuten

Mit geeignetem Messgerät erfassen Sie pro Modus beispielsweise 20 aufeinanderfolgende Toggle-Abstände. Der tatsächliche Messanschluss und gemeinsame Massebezug müssen geprüft sein. Falls nur steigende Flanken erfasst werden, dokumentieren Sie stattdessen vollständige Blinkzyklen und passen die Sollgröße an. Ohne Messgerät ist nur die qualitative LED-Beobachtung zulässig.

Konfigurationsangabe	Eigener Eintrag
Board und Firmwarepaket	
CubeMX und STM32-Erweiterung	
Compiler / Optimierung	
M4- und tatsächlicher TIM2-Takt	

Konfigurationsangabe	Eigener Eintrag
PSC / ARR / Interruptkonfiguration Messpunkte / Trigger / Zeitauflösung	

Modus	Anzahl Abstände	Min.	Max.	Mittel	Spanne	Einheit / Status
Baseline						
Polling konstant						
Polling variabel						
Timer						

Die Tabelle bleibt bis zur Messung leer. Status beispielsweise „gemessen unter angegebenen Bedingungen“, „nur qualitativ beobachtet“ oder „nicht durchgeführt“. Beantworten Sie:

- Welcher Anteil erklärt einen vergrößerten mittleren Flankenabstand, welcher eine Schwankung?
- Weshalb kann ein Timer-Callback während langer Hauptarbeit reagieren?
- Welche Verzögerungen bleiben möglich, und weshalb beweist der Versuch keine harte Zeitgarantie?

Musterlösung Programmvergleich

Die Baseline enthält bereits Wartefunktions-, Tick- und Aufrufeffekte. Konstante Zusatzarbeit vergrößert im idealisierten Modell den Flankenabstand; allein daraus folgt noch kein Periodenjitter. Wechselnde Last kann unterschiedliche Abstände verursachen. Welche Spanne tatsächlich vorliegt, muss gemessen werden. Der Timer zählt unabhängig vom Ende der Hauptschleifenarbeit und kann diese bei zulässigem Interrupt unterbrechen. Maskierung, dringendere Handler, Bus-/Speicherlast und die Handlerkette bleiben als Verzögerungen relevant.

Im Timer-Modus zählt `lab01_diag.timer_callbacks` die behandelten Callback-Aufrufe, nicht garantiert alle Hardwareupdates. Bei zu langsamer Bedienung können Ereignisse zusammenfallen. Zwei Toggles bilden einen LED-Zyklus, sodass 500-ms-Updateabstand ideal einen 1-Hz-Blinkzyklus ergibt. Eine optisch stabile LED oder 20 günstige Perioden beweisen weder eine sichere WCET-Schranke noch die maximale Antwortzeit für alle zugelassenen Lasten. Keine beispielhaften Zeitwerte in die leeren Tabellen eintragen. Ein fehlender Aufbau bleibt als „nicht durchgeführt“ markiert.